



PROVE AN IAM PERMISSIONS BOUNDARY

By Ahmed Tetteh | September 2026

IAM Boundary Decision Proof

Audience: Security lead

Simulator-confirmed predictions: 8/8

Case	Decision rule	Actual result
Five privileged role mutations	Matching explicit Deny	explicitDeny
iam:ListUsers	No matching identity Allow	implicitDeny
iam:CreatePolicyVersion	AllowedByPermissionsBoundary false	implicitDeny
iam:GetRole	Identity Allow and boundary Allow	allowed

Limits and tradeoffs

The policy simulator does not make a live service request, and AWS states that results can differ from the live environment.

No resource-based policy was supplied. No service control policy, cross-account trust, or separate human console session was tested.

The proof trades those wider runtime questions for a reproducible, no-side-effect decision over the policy inputs captured in this repository.

Proving the Permissions Boundary Held the Security Ceiling

Applying and validating the approved boundary

In this step, I'm validating whether the permissions boundary when applied can block off widening of permissions by the IAM role.

Verifying the role's boundary configuration

I verified the boundary by checking role's metadata in json format and saw the arn of the permissions boundary I attached to it within the `PermissionsBoundary` block.

Separating explicit deny, implicit deny, boundary refusal, and allowed access

I distinguished the decisions by checking the effect statements in each action block, and comparing against each other. An explicit Deny in AWS always overrides any Allow. For an allow action, both policies must allow that specific action. For the rest, either they were stated in one policy and not the other, or the action exceeds the permissions boundary ceiling, making it an implicit deny or boundary deny.

Demonstrating the Unbounded Widening Risk

Building the controlled IAM policy and role

In this step, I'm building the boundary policy and role, so that I can verify whether the local widening can grant iam:CreatePolicyVersion with no approved permissions boundary, to give a before and after boundary permission state.

What the allowed pre-boundary result establishes

I observed the "EvalDecision : allowed". This means the role is able to execute the action "iam:CreatePolicyVersion" without any bounded control attached.

```
results > {} 02-baseline.json > ...
1  {
2    "EvaluationResults": [
3      {
4        "EvalActionName": "iam:CreatePolicyVersion",
5        "EvalResourceName": "arn:aws:iam::007399901516:policy/BoundaryProofCeiling",
6        "EvalDecision": "allowed",
7        "MatchedStatements": [
8          {
9            "SourcePolicyId": "PolicyInputList.1",
10           "SourcePolicyType": "IAM Policy",
11           "StartPosition": {
12             "Line": 1,
13             "Column": 130
14           },
15           "EndPosition": {
16             "Line": 1,
17             "Column": 237
18           }
19         }
20       ],
21       "MissingContextValues": [],
22       "EvalDecisionDetails": {},
23       "ResourceSpecificResults": [
24         {
25           "EvalResourceName": "arn:aws:iam::007399901516:policy/BoundaryProofCeiling",
26           "EvalResourceDecision": "allowed",
27           "MatchedStatements": [
28             {
29               "SourcePolicyId": "PolicyInputList.1",
30               "SourcePolicyType": "IAM Policy",
31               "StartPosition": {
32                 "Line": 1,
```

Sealing the Evidence with Verified Teardown

Removing temporary IAM resources in dependency order

In this step, I'm removing the permissions boundary, so that I can delete the boundary policy attached to the role and the other temporary AWS IAM resources.

Why fresh NoSuchEntity reads prove resource absence

The reads prove absence because both resource were completed deleted from AWS.

```
king@king-TFG5577XG:~/Desktop/Personal-Lab/Build & Brew Projects/iam-boundary-proof$ aws iam get-role --role-name BoundaryProofRole --output json --no-cli-pager
aws iam get-policy --policy-arn arn:aws:iam::007399901516:policy/BoundaryProofCeiling --output json --no-cli-pager
aws: [ERROR]: An error occurred (NoSuchEntity) when calling the GetRole operation: The role with name BoundaryProofRole cannot be found.
aws: [ERROR]: An error occurred (NoSuchEntity) when calling the GetPolicy operation: Policy arn:aws:iam::007399901516:policy/BoundaryProofCeiling was not found.
```

```
king@king-TFG5577XG:~/Desktop/Personal-Lab/Build & Brew Projects/iam-boundary-proof$ git add .
king@king-TFG5577XG:~/Desktop/Personal-Lab/Build & Brew Projects/iam-boundary-proof$ git commit -m "evidence: prove teardown and seal security handoff"
git tag -a v1.0.0 -m "IAM permissions boundary proof complete"
git log --oneline --decorate
git remote
[master eef81ba] evidence: prove teardown and seal security handoff
3 files changed, 30 insertions(+), 18 deletions(-)
create mode 100644 docs/after-action-review.md
fatal: tag 'v1.0.0' already exists
eef81ba (HEAD -> master) evidence: prove teardown and seal security handoff
72c52d7 (tag: v1.0.0) test: adjudicate eight frozen predictions
86ad93b build: create frozen IAM control case
25cfb4d docs: freeze security brief design and predictions
2ae74a3 chore: initialize security evidence workspace
```

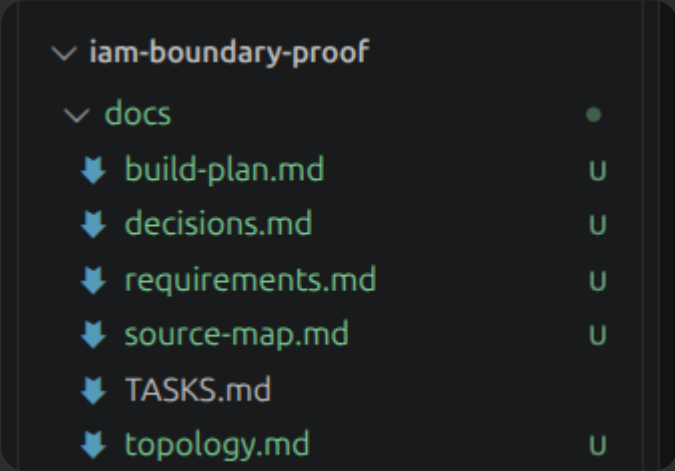
Freezing an Auditable Security Decision

Capturing requirements, policy inputs, and the evidence plan

In this step, I'm freezing the brief (files) that becomes the build instructions before any IAM resources are created.

Preventing predictions from changing after results are known

I froze the predictions first so that they can be used as later score evidence (testable ground truth), i.e, the predictions serve as expected results to validate against the simulator's actual responses.



```

  ✓ iam-boundary-proof
    ✓ docs
      ↓ build-plan.md      U
      ↓ decisions.md      U
      ↓ requirements.md   U
      ↓ source-map.md     U
      ↓ TASKS.md
      ↓ topology.md      U

```

A screenshot of a file explorer window showing a directory structure. The root directory is 'iam-boundary-proof', which is expanded to show a subdirectory 'docs'. The 'docs' directory is also expanded, showing a list of files: 'build-plan.md', 'decisions.md', 'requirements.md', 'source-map.md', 'TASKS.md', and 'topology.md'. Each file has a blue arrow icon to its left and a green 'U' icon to its right, indicating that the files are up-to-date or frozen.

Establishing a Trusted Local Evidence Workspace

Verifying a non-root administrative session and local repository

In this step, I'm verifying the evidence that a security lead needs for an accountable AWS Identity and Access Management (IAM) caller by preparing the workspace needed needed for the design and trailing.

```
king@king-TF65577XG:~/Desktop/Personal-Lab/Build & Brew Projects/iam-boundary-proof$ git log --oneline
2ae74a3 (HEAD -> master) chore: initialize security evidence workspace
king@king-TF65577XG:~/Desktop/Personal-Lab/Build & Brew Projects/iam-boundary-proof$ git remote
```

Framing the Security Lead's Question

Can the local widening grant CreatePolicyVersion after the boundary is applied?

I am testing whether local evidence package that answers whether a proposed policy widening can escape an approved limit in AWS Identity and Access Management (IAM) using permission boundaries.

Reflecting on IAM Evidence Engineering

Tools and authorization concepts applied

The key tools I used include AWS IAM, VS Code . Key concepts I learnt include how the permissions boundary can be used to mitigate a lot of security flaws by creating a ceiling around an entity or role to either deny or allow certain actions.

Completion time and implementation challenges

This project took me approximately 90 mins. The most challenging part was trying to pass in the widened policy(--policy-input-list) as a JSON string of policy documents, and not as a JSON object, as is normally written.

Next steps in security and cloud engineering

I did this project today to learn how to Cloud security personnels are able to evaluate IAM policies to verify that the roles attached to those policies do not grant unintended permissions to an entity, and the permissions only stay within a boundary.